

COMP 1324 Algorithmics

Greedy Strategy

Hikmat Farhat

March 17, 2026

Introduction

- Greedy algorithms are algorithms that make **locally** optimal choices at each step.
- The **hope** is that such choices will lead to a **globally** optimal solution.
- That is not always the case but when it is, greedy algorithms are very **efficient**.
- We have already seen some examples of greedy algorithms: Dijkstra's, Prim's, Kruskal's.
- We will look at some more examples of greedy algorithms.

Job Scheduling

- Assume we have n jobs, each with weight w_i and length l_i , $1 \leq i \leq n$.
- The jobs share some resource (i.e. CPU) so they must be run sequentially.
- If we run the jobs in the order $1, 2, 3, \dots$ then job i has completion time $c_i = \sum_{k=1}^i l_k$.
- our goal is to minimize the quantity $f = \sum_{k=1}^n w_k \cdot c_k$
- In particular, we would like to have a greedy algorithm that minimizes f .
- To do that we start by looking at special cases

Special case 1

- In this special case we assume that all jobs have the same weight then

$$\begin{aligned} f &= w(l_1 + (l_1 + l_2) + (l_1 + l_2 + l_3) + \dots + (l_1 + \dots + l_n)) \\ &= w(n \cdot l_1 + (n-1) \cdot l_2 + (n-2) \cdot l_3 + \dots + 1 \cdot l_n) \end{aligned}$$

- Clearly f will be minimized if we choose $l_1 \leq l_2 \leq \dots \leq l_n$
- This situation occurs often enough to warrant its own name:
- Shortest job (task) first.

Special case 2

- The second special case is when all jobs have the same length l then

$$f = l(w_1 + 2 \cdot w_2 + 3 \cdot w_3 + \dots n \cdot w_n)$$

- Clearly, f will be minimized if $w_1 \geq w_2 \geq w_3 \geq \dots \geq w_n$
- it is clear that f decreases by choosing the **largest** w or the **smallest** l first.
- Can we infer the general case from these two special cases?

The general case

- It can be shown (using an exchange argument) that choosing, among the remaining tasks, the one with **largest** ratio w_i/l_i first, leads to the optimal solution.

```

input: An array  $A$  of  $n$  pairs  $(w_i, l_i)$ 
for  $i \leftarrow 1$  to  $n$  do
    |  $B[i] \leftarrow A(w_i/l_i, i);$ 
 $B \leftarrow \text{sort}(B);$ 
foreach  $(w/l, k) \in B$  do
    |  $\text{run}(k);$ 
    
```

- The algorithm runs in $O(n \log n)$ time.

Interval Scheduling

- Consider a set of n intervals (s, e) where s and e are the starting and ending time respectively.
- We would like to choose a non-overlapping subset of those intervals such that the total number of selected intervals is maximum.
- For example, consider the intervals $\{(1, 5), (2, 7), (5, 8)\}$. The largest subset of non-overlapping intervals is $\{(1, 5), (5, 8)\}$.

We are looking for a greedy solution to this optimization problem.
What property of the intervals should the greedy method select?

There are many options:

- 1 shortest interval first
- 2 The interval with the smallest starting time
- 3 The interval with the smallest number of overlaps
- 4 etc...

Shortest interval first



Figure: Shortest interval counterexample

Earliest starting time first



Figure: counterexample for earliest starting time first

Smallest overlap first



Figure: Smallest overlap first

Greedy Solution

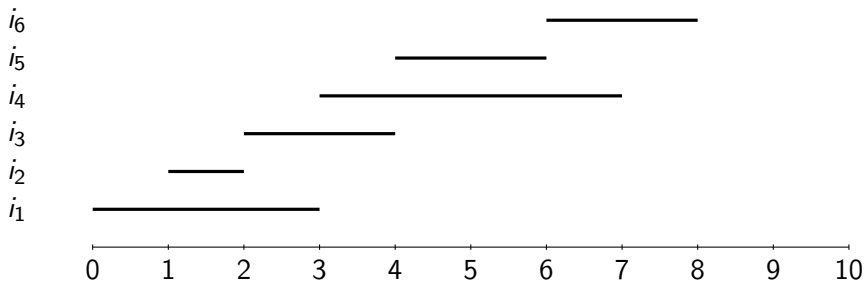
The greedy solution consists of choosing the next compatible interval with the smallest finishing time. We build a min-heap based on finishing times. Let I be the set of intervals and T the desired solution

```

Q ← I
T ← ∅
last ← -1
while Q ≠ ∅ do
    (s, f) ← EXTRACT-MIN(Q)
    if s ≥ last then
        T ← T ∪ {(s, f)}
        last ← f

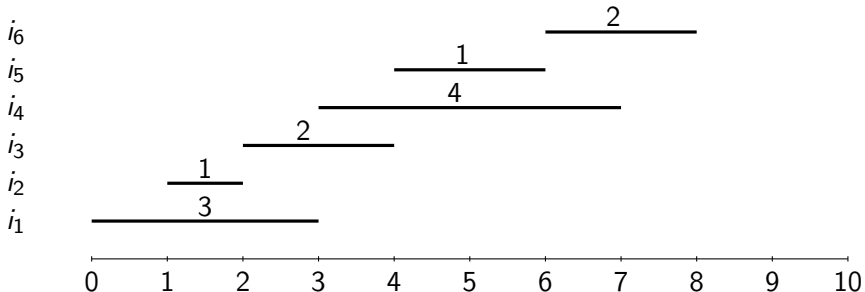
```

Example



- Choosing smallest finishing time first leads to the solution $\{i_2, i_3, i_5, i_6\}$ which is optimal.

Intervals with weights



- If we associate a value with each interval as shown above, and seek the set of intervals with maximum value then the greedy strategy does not work.

Fractional Knapsack

- Given n items having value $v_1, \dots, v_n$ and weights $w_1, \dots, w_n$ and a knapsack of capacity C
- We need to maximize

$$\sum_{i=1}^n x_i \cdot v_i$$

- Subject to the condition

$$\sum_{i=1}^n x_i \cdot w_i \leq C$$

- Where $0 \leq x_i \leq 1$ is a fraction of item i that is used.

Greedy Solution

- Add item with biggest ratio v/w until the next item does not "fit" in knapsack so we add a fraction of it.
- Either sort items by v_i/w_i or add them to a max-heap with v_i/w_i as key.

$W \leftarrow C$

while $W > 0 \wedge Q \neq \emptyset$ **do**

$(i, v_i, w_i) \leftarrow \text{EXTRACT-MAX}(Q)$

if $w_i \leq W$ **then**

$W \leftarrow W - w_i$

$x_i \leftarrow 1$

else

$x_i \leftarrow w_i / W$

$W \leftarrow 0$

Example

- Consider a knapsack with capacity $W = 36$ and the following items

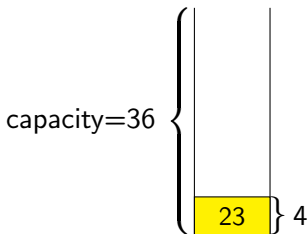
item	1	2	3	4	5	6	7	8
value	9	2	8	2	19	13	20	23
weight	11	16	19	14	18	20	22	4

- First step is to compute the ratio v_i/w_i and sort the items in decreasing order of the ratio.

item	8	5	7	1	6	3	4	2
ratio	5.75	1.06	0.91	0.82	0.65	0.42	0.14	0.12
value	23	19	20	9	13	8	2	2
weight	4	18	22	11	20	19	14	16

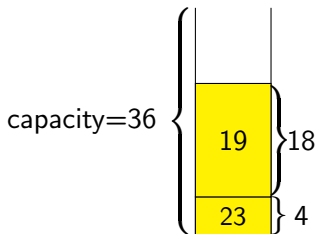
Example

item	8	5	7	1	6	3	4	2
ratio	5.75	1.06	0.91	0.82	0.65	0.42	0.14	0.12
value	23	19	20	9	13	8	2	2
weight	4	18	22	11	20	19	14	16



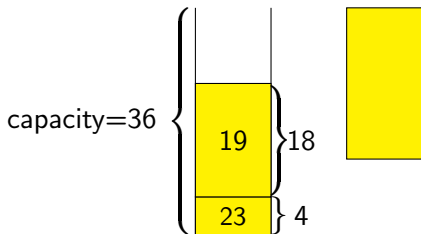
Example

item	8	5	7	1	6	3	4	2
ratio	5.75	1.06	0.91	0.82	0.65	0.42	0.14	0.12
value	23	19	20	9	13	8	2	2
weight	4	18	22	11	20	19	14	16



Example

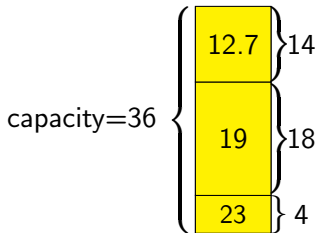
item	8	5	7	1	6	3	4	2
ratio	5.75	1.06	0.91	0.82	0.65	0.42	0.14	0.12
value	23	19	20	9	13	8	2	2
weight	4	18	22	11	20	19	14	16



Item 7 does not fit

Example

item	8	5	7	1	6	3	4	2
ratio	5.75	1.06	0.91	0.82	0.65	0.42	0.14	0.12
value	23	19	20	9	13	8	2	2
weight	4	18	22	11	20	19	14	16



A fraction (0.6363) of Item 7 fits

0-1 Knapsack

- In the 0-1 knapsack problem we are not allowed to take fractions of items. The greedy strategy does not always work in this case. Consider the example below where the knapsack capacity is 13:

item	1	2	3	4
ratio	0.66	0.66	0.71	0.75
value	2	4	5	6
weight	3	6	7	8

- The greedy algorithm would choose item 4 first. After that only item 1 would fit in the knapsack for a total value of 8. The optimal solution is to choose items 2 and 3 for a total value of 9.
- If fractional items are used we get an upper bound of 11.57

Symbol encoding

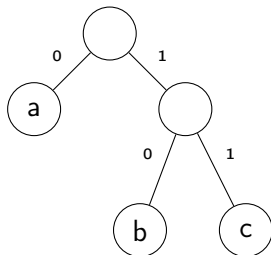
- Suppose we have an alphabet $S = \{s_1, \dots, s_k\}$ of size k and we want to encode a message M consisting of n symbols from S .
- A trivial encoding is to use $n \cdot \lceil \log_2 k \rceil$ bits to encode the message.
- For example, $S = \{a, b, c\}$ and $M = abaabc$ then we can assign $a = 00, b = 01$ and $c = 10$ and thus
- $M = 00\ 01\ 00\ 00\ 01\ 10$ (space added for clarity)
- For a total of 12 bits.

Can we do better?

- Is it possible to encode the message using less than 12 bits?
- The answer is yes. We can use a variable length encoding.
- The trick is to use shorter codes for the most frequent symbols and longer codes for the least frequent symbols.
- But then we lose the "boundary" between symbols and creates ambiguity.
- Solution is to use **prefix** codes.
- This means there is no code that is a prefix of another code.
- For example if $a = 0$ and $b = 01$ then a is a prefix of b and this is not allowed.

Example prefix code

- Consider the previous example $S = \{a, b, c\}$ and $M = abaabc$ with $a = 0$, $b = 10$, $c = 11$. $M = 0\ 10\ 0\ 0\ 10\ 11$ for a total of 9 bits instead of 12.
- Note that prefix code can be represented by a binary tree.



Huffman Coding

- Formally, given an alphabet $S = \{s_1, \dots, s_k\}$ with frequencies $f_1, \dots, f_k$ we want to find a prefix code such that the average number of bits per symbol is minimized.
- Using the prefix tree representation, the average number of bits is given by

$$A = \sum_{i=1}^k f_i \cdot d(s_i)$$

where $d(s_i)$ is the depth of the leaf corresponding to s_i .

- Huffman coding is a greedy algorithm that constructs the prefix tree "bottom up".

- First a prefix code is equivalent to a binary tree so once we build the optimal binary tree then we "read off" the encoding.
- The basic idea of HC is to build the optimal tree recursively in a greedy manner
 - ① The optimal tree T for k symbols is obtained by constructing the optimal tree T' for $k - 1$ symbols where T' is the same as T except replacing the two nodes with the smallest frequencies in T , x and y by a single node having the sum of the frequencies: $f_w = f_x + f_y$

$$T' = T - \{x, y\} \cup \{w\}$$

The optimal tree has the following properties:

- It is full. Suppose that y is a single child of node w . By replacing w by y we obtain a "better" tree
- For any two leaves x, y if $f_x > f_y$ then $d_x < d_y$. This can be shown by an exchange argument.

Note that there could be many optimal trees. Let x, y be the symbols with the least frequencies then there exists an optimal tree in which x, y are siblings.

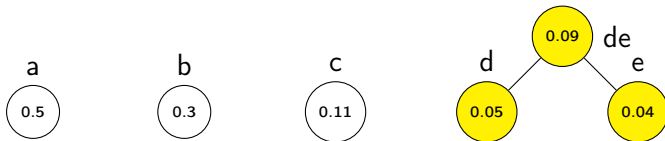
Huffman Coding Example

(0.5, 'a'), (0.3, 'b'), (0.11, 'c'), (0.05, 'd'), (0.04, 'e')



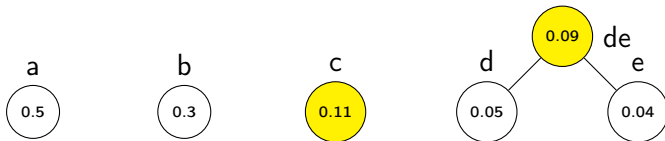
Huffman Coding Example

(0.5, 'a'), (0.3, 'b'), (0.11, 'c'), (0.09, 'de')



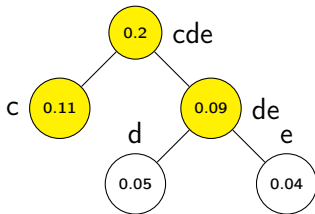
Huffman Coding Example

$(0.5, 'a')$, $(0.3, 'b')$, $(0.11, 'c')$, $(0.09, 'de')$



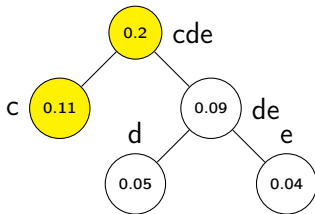
Huffman Coding Example

$(0.5, 'a')$, $(0.3, 'b')$, $(0.2, 'cde')$



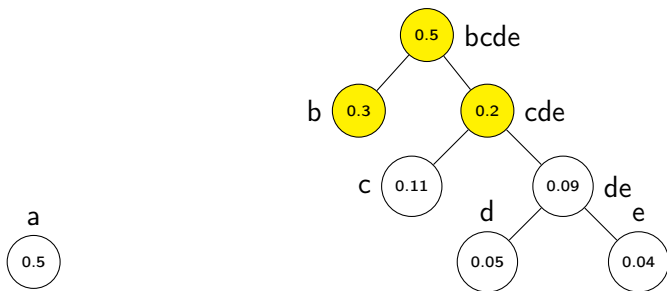
Huffman Coding Example

$(0.5, 'a')$, $(0.3, 'b')$, $(0.2, 'cde')$



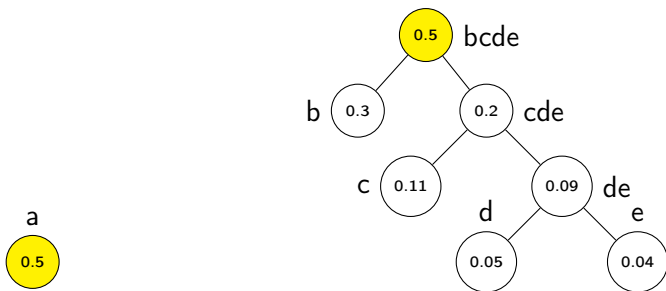
Huffman Coding Example

$(0.5, 'a')$, $(0.5, 'bcde')$



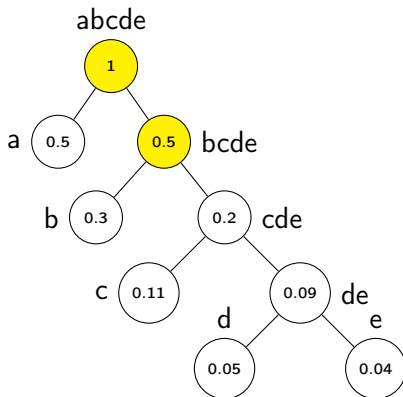
Huffman Coding Example

$(0.5, 'a')$, $(0.5, 'bcde')$



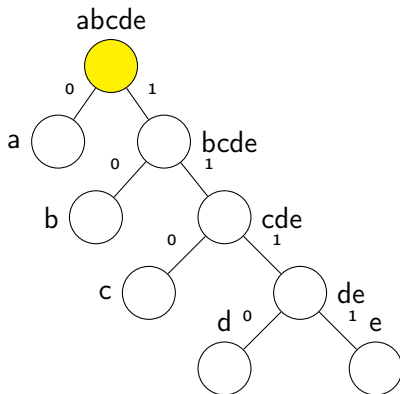
Huffman Coding Example

(1, 'abcde')



Huffman Coding Example

a	0
b	10
c	110
d	1110
e	1111



Huffman coding: pseudocode

Input: Array of frequency, symbol pairs, F

Result: Huffman tree

```
function BuildTree ( $F$ )  
    for  $(f, s) \in F$  do  
        INSERT ( $Q$ , new Node( $f, s$ ))  
    while  $|Q| > 1$  do  
         $x \leftarrow \text{EXTRACT-MIN}(Q)$   
         $y \leftarrow \text{EXTRACT-MIN}(Q)$   
         $z \leftarrow \text{new node}$   
         $z.\text{left}, z.\text{right} \leftarrow x, y$   
         $z.f \leftarrow x.f + y.f$   
        INSERT ( $Q, z$ )  
    return EXTRACT-MIN ( $Q$ )
```

Huffman coding: pseudocode

input: root of the Huffman tree

Result: The code for each symbol

WalkTree (root,"")

function WalkTree (*node*,*prefix*)

if *node* is a leaf **then**

 code[*node*]=*prefix*

else

 WalkTree (*node*.left,*prefix*+"0")

 WalkTree (*node*.right,*prefix*+"1")
